OOP & Design — Interview Prep (Simple Version)

SOLID Principles

S — Single Responsibility A class should do ONE thing only. Bad: `User` class that saves to DB, sends email, AND validates data. Good: split into `User`, `UserRepository`, `EmailService`, `UserValidator`. *Why asked:* easier to maintain, one reason to change.

O — Open/Closed Class should be open for extension, closed for modification. Don't edit existing code to add new behavior — extend it instead.

```
interface Discount { double apply(double price); }
class NewYearDiscount implements Discount { ... }
// Add new discount type without touching old code
```

L — Liskov Substitution Subclass should be replaceable for parent class without breaking things. Classic bad example:

```
class Bird { void fly() {} }
class Ostrich extends Bird { void fly() { throw new Error(); } } // BREAKS LSP
```

Ostrich can't fly, but it's forced to have `fly()`. Fix: don't put `fly()` in base `Bird`.

I — Interface Segregation Don't force a class to implement methods it doesn't need. Bad: one fat `Worker` interface with `code()`, `eat()`, `sleep()` — a Robot has to implement `eat()` too. Good: split into smaller interfaces — `Workable`, `Eatable`.

D — Dependency Inversion Depend on interfaces, not concrete classes.

```
class Service {
    private Database db; // bad: tightly coupled to one DB
    private DatabaseInterface db; // good: can swap MySQL/Postgres/Mock
}
```

This is also what makes Spring's dependency injection work.

Quick memory trick: *"A class Should be Open, but Liskov says don't Inherit badly, keep Interfaces small, and Depend on abstractions."*

Design Patterns

Singleton

Only ONE instance of a class should exist (e.g., a config manager, connection pool).

```
class Singleton {
    private static Singleton instance;
    private Singleton() {} // private constructor blocks "new"
    public static Singleton getInstance() {
        if (instance == null) instance = new Singleton();
        return instance;
    }
}
```

Tricky part interviewers ask: this is NOT thread-safe. Two threads could both pass the `null` check at the same time and create two instances. Fix — thread-safe version:

```
public static synchronized Singleton getInstance() { ... }
// or better: "double-checked locking" or an enum Singleton
```

Simplest thread-safe way in real interviews: use an `enum` — Java guarantees it's instantiated once.

Factory

A method that creates objects for you, so calling code doesn't need to know the exact class.

```
interface Shape {}
class Circle implements Shape {}
class Square implements Shape {}

class ShapeFactory {
    Shape create(String type) {
        if (type.equals("circle")) return new Circle();
        return new Square();
    }
}
```

Why used: calling code just says `factory.create("circle")` — doesn't need to know `Circle` exists directly. Easy to add new shapes later.

Builder

Used when an object has MANY optional fields — avoids messy constructors.

```
Pizza pizza = new Pizza.Builder()
                .size("large")
                .cheese(true)
                .pepperoni(true)
                .build();
```

Why used: avoids "constructor with 8 parameters" problem (telescoping constructors). Very common with immutable objects.

Observer

One object (subject) notifies many others (observers) when something changes. Real example: YouTube channel (subject) → subscribers (observers) get notified on new video.

```
interface Observer { void update(String event); }
class Subject {
    List<Observer> observers = new ArrayList<>();
    void notifyAll(String event) {
        for (Observer o : observers) o.update(event);
    }
}
```

Where it's used in real life: Spring's `ApplicationEvent`, message queues, listener patterns in UI frameworks.

Strategy

Define a family of algorithms, make them swappable at runtime. Real example: payment methods (CreditCard, PayPal, UPI) — same interface, different logic.

```
interface PaymentStrategy { void pay(int amount); }
class CreditCard implements PaymentStrategy { ... }
class UPI implements PaymentStrategy { ... }

class Cart {
    PaymentStrategy strategy;
    void checkout(int amount) { strategy.pay(amount); }
}
```

Why used: avoids big if-else chains for different behaviors. Behavior can change at runtime by swapping the strategy object.

Quick pattern cheat sheet

Pattern	Solves this problem
Singleton	Need exactly one instance
Factory	Hide object creation logic from caller
Builder	Too many constructor parameters
Observer	One-to-many notification
Strategy	Swap algorithms/behavior at runtime

Composition vs Inheritance

Inheritance = "IS-A" relationship. `Dog extends Animal`. **Composition** = "HAS-A" relationship. `Car has an Engine`.

Why composition is usually preferred:

1. Inheritance locks you into a rigid hierarchy at compile time.
2. Composition lets you swap behavior at runtime (just plug in a different object).
3. Inheritance can break if the parent class changes internally — subclass depends on parent's guts.

Simple example:

```
// Inheritance — rigid
class Car extends Engine {} // weird, a Car "is-a" Engine? No.

// Composition — correct
class Car {
    private Engine engine; // Car "has-a" Engine
}
```

Real interview line to remember: *"Favor composition over inheritance"* — use inheritance only for a true IS-A relationship, otherwise use composition.

Fast Recall Table

Concept	One-line hook
SRP	One class, one job
OCP	Extend, don't edit
LSP	Subclass shouldn't break parent's promise
ISP	Don't force unused methods
DIP	Depend on interface, not class
Singleton	One instance only
Factory	Hides object creation
Builder	Fixes too-many-constructor-params
Observer	Notify many on change
Strategy	Swap algorithm at runtime
Composition	HAS-A, flexible
Inheritance	IS-A, rigid